MBLP131 · Hafta 12 · Ders Notları

# İşletim Sistemleri Temelleri

Metin İşleme, Yönlendirme ve Boru Hatları

*Çekirdeğin birbirine bağladığı küçük programlar, gerçek veriler hakkındaki gerçek soruları yanıtlar.*

**Öğretim üyesi:** Dr. Öğr. Üyesi Hasan Oğuz
Elektronik Haberleşme Teknolojisi Program Başkanı, Meslek Yüksekokulu
İstanbul Okan Üniversitesi

### Bu haftanın sonunda yapabilmeniz gerekenler

1. Standart girdiyi, standart çıktıyı ve standart hatayı $0$, $1$ ve $2$ numaralı dosya tanımlayıcıları olarak açıklamak ve bunları 2–3. haftalardaki `write(1, …)` sistem çağrısına bağlamak.
2. `>`, `>>`, `<`, `2>`, `2>&1`, `&>` yönlendirmelerini ve `/dev/null` aygıtını kullanmak; sıralarının etkisini önceden kestirmek.
3. Kabuğun yönlendirmeyi ve boruları `fork()`, `open()`, `pipe()`, `dup2()` ve `exec()` ile nasıl gerçeklediğini açıklamak ve ilgili `strace` çıktısını okumak.
4. `grep` (`-i -n -v -c -w -r` seçenekleri ve basit düzenli ifadelerle), `sort`, `uniq`, `cut`, `wc`, `tr`, `head`, `tail` ve `tee` filtrelerini kullanmak.
5. Dizin ağacında `find` ile arama yapmak ve sonuçlar üzerinde `-exec` ya da `xargs` ile işlem yapmak.
6. Günlük (log) dosyaları ve CSV verileri hakkındaki soruları, her seferinde bir aşama eklenerek kurulan tek satırlık boru hatlarıyla yanıtlamak.

## İçindekiler

1 11. Haftadan 12. Haftaya 1

2 Standart Akışlar ve Dosya Tanımlayıcıları 2

3 Yönlendirme 4

4 Borular 6

5 Satır Aramak: `grep` 9

6 Sıralamak ve Saymak: `sort` ve `uniq` 11

7 Sütunlar, Sayımlar ve Karakterler: `cut`, `wc`, `tr` 13

8 Diğer Araçlar: `head`, `tail`, `tee`, `find`, `xargs` 14

9 Tek Satırlık Boru Hatlarıyla Gerçek Analizler 16

10 Komut Başvuru Tablosu 19

11 Sık Yapılan Hatalar ve Anlamları 20

12 Lab Föyü 21

13 Özet 22

14 Sözlük 22

15 Kendiniz Deneyin 23

16 Tekrar Soruları 23

17 Lab Föyü Çözümleri 24

18 İleriye Bakış: 13. Hafta 25

# 1 11. Haftadan 12. Haftaya

Geçen hafta dosyalara dışarıdan baktık: sahipleri kim (`id`, `/etc/passwd`), `-rwxr-x---` kip dizgesi ne anlama geliyor ve `chmod` bunu sembolik (`u+x`) ya da sekizli olarak nasıl değiştiriyor ($754$: sahip için $7 = 4+2+1$, grup için $5 = 4+1$, diğerleri için $4$). Bu hafta dosyaların *içine* bakıyoruz. Linux neredeyse her şeyi — yapılandırmayı, günlükleri, veri dökümlerini — satır başına bir kayıt olacak biçimde düz metin olarak tutar. Bu yüzden satır okuyan, satırları süzen ve satır yazan küçük bir araç takımı, şaşırtıcı çeşitlilikte soruyu yanıtlayabilir. Bu araçları birbirine bağlayan yapıştırıcıyı işletim sistemi sağlar: **yönlendirme** (redirection) ve **borular** (pipes). İkisi de daha önce karşılaştığınız tek bir fikre dayanır: **dosya tanımlayıcı** (file descriptor).

**Bu notları nasıl izlemelisiniz?** Tüm örnekler `~/lab` alıştırma klasöründe çalışır. Dosyalarınızın bizimkilerle aynı olması için başlamadan önce klasörü yeniden oluşturun:

```
$ bash mcmp117_lab_setup.sh
$ cd ~/lab
```

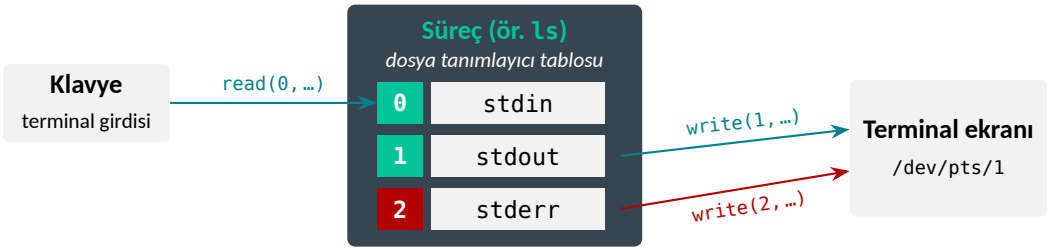
Burada gösterilen her oturum gerçek çıktıdır; sıradan bir kullanıcı olarak `code/demo_*.sh` betikleriyle yakalanmıştır (çıktı dosyaları `code/out_*.txt`). Sizin makinenizde tarihler, süreç numaraları (PID), terminal adları (`/dev/pts/1`) ve kullanıcı adları farklı olacaktır. `ls` ve `sort` çıktısının sırası da *yerel ayara* (locale, dil ayarları) bağlı olabilir; biz büyük harflerin küçük harflerden önce sıralandığı `LC_ALL =C.UTF-8` ayarını kullandık. Türkçe yerel ayarlı bir sistemde (`LANG =tr_TR.UTF-8`) pek çok ileti Türkçe görünür ve sıralama farklı olabilir; sonuçlarınız farklı sıralanıyorsa `export LC_ALL =C.UTF-8` yazın. Programların ilettiği metinler (ör. `No such file or directory`) burada yakalandıkları gibi İngilizce bırakılmıştır.

# 2 Standart Akışlar ve Dosya Tanımlayıcıları

## 2.1 Doğuştan açık üç dosya

7. haftada bir sürecin açık bir dosyaya adıyla değil, küçük bir tamsayıyla, yani **dosya tanımlayıcıyla** (file descriptor, fd) başvurduğunu gördük: bu sayı, sürecin *süreç başına açık dosya tablosunda* bir sıra numarasıdır. Kabuk bir programı başlattığında üç tanımlayıcı zaten açıktır:

| fd | Ad | Kullanım | Terminalde varsayılan |
|---|---|---|---|
| 0 | standart girdi (stdin) | programın okuduğu veri | klavye |
| 1 | standart çıktı (stdout) | normal sonuçlar | terminal ekranı |
| 2 | standart hata (stderr) | hata ve tanı iletileri | terminal ekranı |




**Şekil 1:** Üç standart akış. Çıktı ve hatalar, varsayılan olarak aynı ekranda biten iki ayrı kanaldır.

Bunları kim açtı? Kabuk; daha doğrusu kabuğu başlatan süreçler zinciri. `fork()` ile oluşturulan bir çocuk süreç, ebeveyninin tanımlayıcı tablosunun bir kopyasını alır ve `exec()` bu tabloyu korur (3. hafta). Dolayısıyla `bash`, `ls` komutunu çalıştırdığında `ls`, `bash`'in hepsi terminali gösteren $0$, $1$, $2$ tanımlayıcılarını miras alır.

Neden iki çıktı kanalı var? `ls`'in listesini bir dosyaya kaydettiğini ve argümanlarından birinin var olmadığını düşünün. Hata iletisi listeye karışsaydı dosyanın içinde gizli kalırdı. Hatalar kendi tanımlayıcılarında yol aldığı için yine de ekrana ulaşırlar. Bu basit tasarım kararı, 2> yönlendirmesinin var olma nedenidir.

## 2.2 Bunu daha önce gördünüz: `write(1, …)`

2. haftada `strace`, `printf("Hello…")` çağrısının sonunda `write(1, "Hello…", n)` sistem çağrısına dönüştüğünü gösterdi; 3. haftada aynı çağrıyı `syscall` komutuyla elle yaptık: ilk parametre olan $1$, standart çıktıydı. Şimdi `ls`'i bir var olan ve bir eksik argümanla izleyelim (`-o t1.txt`, izi bir dosyaya yazar; böylece iz `ls`'in çıktısına karışmaz):

```
student@okan-lab:~/lab$ strace -o t1.txt -e trace=write ls data nosuch
ls: cannot access 'nosuch': No such file or directory
data:
cities.txt  numbers.txt  students.csv
student@okan-lab:~/lab$ cat t1.txt
write(2, "ls: ", 4)                      = 4
write(2, "cannot access 'nosuch'", 22)  = 22
write(2, ": No such file or directory", 27) = 27
write(2, "\n", 1)                        = 1
```

```
write(1, "data:\n", 6)                  = 6
write(1, "cities.txt  numbers.txt  student"..., 38) = 38
+++ exited with 2 +++
```

Hata iletisinin dört parçası `write(2, …)` ile, liste ise `write(1, …)` ile çıkar. `ls` programı $1$ ve $2$ numaralı tanımlayıcıların *nereye* gittiğini hiç sormaz; bu, program başlamadan önce belirlenmiştir. Kabuk $1$ numaralı tanımlayıcıyı bir dosyaya yöneltirse her `write(1, …)` o dosyaya düşer. Yönlendirmenin bütün sırrı budur.

**Küçük bir sürpriz.** Terminalde `ls`, `cities.txt  numbers.txt  students.csv` adlarını sütunlar hâlinde yazdı; ama iz (ve aşağıda içine yönlendirdiğimiz her dosya) satır başına bir ad içerir. `ls`, `isatty(1)` kütüphane işleviyle $1$ numaralı tanımlayıcının bir terminal olup olmadığını denetler ve biçimini buna göre seçer. Programların çoğu bununla ilgilenmez; birkaçı (`ls`, `grep --color=auto`) ilgilenir.

## 2.3 Tanımlayıcıları görmek: `/proc/self/fd`

`/proc` dosya sistemi (3. hafta) çekirdeğin her sürece bakışını gösterir. `/proc/self`, bakan sürecin kendisidir; `/proc/self/fd` ise onun tanımlayıcılarını sembolik bağlantılar olarak listeler. Bu yüzden `ls -l /proc/self/fd` komutu `ls`'in kendi tanımlayıcılarını gösterir:

```
student@okan-lab:~/lab$ ls -l /proc/self/fd
total 0
lrwx------ 1 student student 64 Oct 12 08:48 0 -> /dev/pts/1
lrwx------ 1 student student 64 Oct 12 08:48 1 -> /dev/pts/1
lrwx------ 1 student student 64 Oct 12 08:48 2 -> /dev/pts/1
lr-x------ 1 student student 64 Oct 12 08:48 3 -> /proc/15829/fd
student@okan-lab:~/lab$ ls -l /proc/self/fd > fd.txt
student@okan-lab:~/lab$ cat fd.txt
total 0
lrwx------ 1 student student 64 Oct 12 08:48 0 -> /dev/pts/1
l-wx------ 1 student student 64 Oct 12 08:48 1 -> /home/student/lab/fd.txt
lrwx------ 1 student student 64 Oct 12 08:48 2 -> /dev/pts/1
lr-x------ 1 student student 64 Oct 12 08:48 3 -> /proc/15832/fd
student@okan-lab:~/lab$ ls -l /proc/self/fd < docs/syllabus.txt
total 0
lr-x------ 1 student student 64 Oct 12 08:48 0 -> /home/student/lab/docs/syllabus.txt
lrwx------ 1 student student 64 Oct 12 08:48 1 -> /dev/pts/1
lrwx------ 1 student student 64 Oct 12 08:48 2 -> /dev/pts/1
lr-x------ 1 student student 64 Oct 12 08:48 3 -> /proc/15838/fd
```

- Normalde $0$, $1$ ve $2$'nin üçü de `/dev/pts/1` terminal aygıtını gösterir. $3$ numaralı tanımlayıcı, `ls`'in o anda okuduğu dizindir.
- `> fd.txt` ile $1$ numaralı tanımlayıcı `/home/student/lab/fd.txt` dosyasını gösterir; `ls` kendi listesini bu dosyaya yazdı. `l-wx------` kipi, dosyanın yalnızca yazma için açık olduğunu gösterir.
- `< docs/syllabus.txt` ile $0$ numaralı tanımlayıcı salt okunur açılmış dosyadır (`lr-x------`).
- `2> /dev/null` ile (bkz. `code/out_streams.txt`) $2$ numaralı tanımlayıcı `/dev/null` aygıtını gösterir.

# 3 Yönlendirme

## 3.1 Çıktı: > ve >>

```
student@okan-lab:~/lab$ echo "first line" > notes.txt
student@okan-lab:~/lab$ cat notes.txt
first line
student@okan-lab:~/lab$ echo "second line" >> notes.txt
student@okan-lab:~/lab$ cat notes.txt
first line
second line
student@okan-lab:~/lab$ echo "oops" > notes.txt
student@okan-lab:~/lab$ cat notes.txt
oops
```

`komut > dosya`, gerekirse dosyayı oluşturur, *içini boşaltır* ($0$ bayta kırpar, truncate) ve $1$ numaralı tanımlayıcıyı ona bağlar. `komut >> dosya` ise dosyayı *sona ekleme* (append) için açar: yeni veri sona gider. Yukarıdaki üçüncü `echo` iki satırın ikisinin de yerini aldı: eski içerik gitti; geri alma da çöp kutusu da yok (10. hafta). Bash sizi koruyabilir: `set -o noclobber` komutundan sonra kabuk, var olan bir dosyanın üzerine > ile yazmayı reddeder (>| işleci bunu zorlar).

## 3.2 Girdi: <

```
student@okan-lab:~/lab$ wc -l logs/access.log
300 logs/access.log
student@okan-lab:~/lab$ wc -l < logs/access.log
300
```

`wc -l logs/access.log` komutunda dosyayı wc kendisi açar ve adını bilir, bu yüzden adı da yazar. `wc -l < logs/access.log` komutunda ise dosyayı $0$ numaralı tanımlayıcı olarak *kabuk* açar ve wc yalnızca standart girdisini okur: yazacak bir adı yoktur. Bu çıplak sayı betiklerde kullanışlıdır (14. hafta).

**Filtre geleneği**

Metin araçlarının çoğu birer **filtredir** (filter): komut satırında adı verilen dosyaları, hiç dosya verilmemişse standart girdiyi okurlar ve sonuçlarını standart çıktıya yazarlar. Hata iletileri standart hataya gider. Bu gelenek sayesinde her filtre bir dosyadan, klavyeden ya da başka bir programdan beslenebilir — ve çıktısı ekrana, bir dosyaya ya da başka bir programa gidebilir.

## 3.3 Hatalar: 2>, 2>&1, &> ve /dev/null

Bir yönlendirmenin genel biçimi n>dosya'dır; burada n bir tanımlayıcı numarasıdır. Tek başına >, 1> demektir; tek başına < ise 0< demektir.

```
student@okan-lab:~/lab$ ls data nosuch > out.txt
ls: cannot access 'nosuch': No such file or directory
student@okan-lab:~/lab$ cat out.txt
data:
cities.txt
numbers.txt
students.csv
```

```
student@okan-lab:~/lab$ ls data nosuch 2> err.txt
data:
cities.txt  numbers.txt  students.csv
student@okan-lab:~/lab$ cat err.txt
ls: cannot access 'nosuch': No such file or directory
student@okan-lab:~/lab$ ls data nosuch > all.txt 2>&1
student@okan-lab:~/lab$ cat all.txt
ls: cannot access 'nosuch': No such file or directory
data:
cities.txt
numbers.txt
students.csv
```

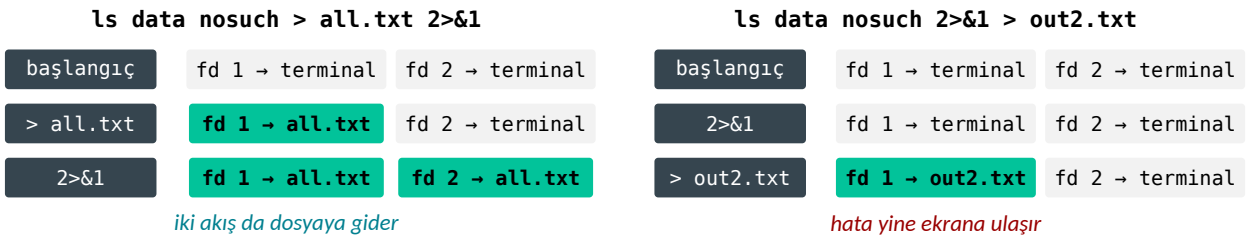
```
student@okan-lab:~/lab$ ls data nosuch &> all.txt
student@okan-lab:~/lab$ ls data nosuch 2> /dev/null
data:
cities.txt  numbers.txt  students.csv
```

- `> out.txt` yalnızca standart çıktıyı taşır; bu yüzden hata yine ekranda görünür.
- `2> err.txt` yalnızca standart hatayı taşır: liste görünür, hata dosyadadır.
- `> all.txt 2>&1` ikisini de dosyaya koyar. `2>&1`, "2 numaralı tanımlayıcıyı 1 numaralının kopyası yap" diye okunur.
- `&> all.txt`, aynı işin bash kısaltmasıdır (daha basit olan `sh`/`dash` kabuklarında çalışmaz).
- `2> /dev/null` hataları atar. `/dev/null` bir *aygıt dosyasıdır*: ona yazılan her şey kaybolur, ondan okumak ise hemen dosya sonu (end-of-file) döndürür.

## 3.4 Sıra önemlidir

Yönlendirmeler *soldan sağa* işlenir ve `2>&1`, 1 numaralı tanımlayıcıyı *o anki hâliyle* kopyalar; kalıcı bir bağ kurmaz. Karşılaştırın:

```
student@okan-lab:~/lab$ ls data nosuch 2>&1 > out2.txt
ls: cannot access 'nosuch': No such file or directory
student@okan-lab:~/lab$ cat out2.txt
data:
cities.txt
numbers.txt
students.csv
```




**Şekil 2:** Soldan sağa işlenen her yönlendirmeden sonra tanımlayıcı tablosu.

### 3.5 Kabuk bunu nasıl yapar?

3. haftada UNIX'in süreçleri neden iki ayrı çağrıyla, `fork()` ve `exec()` ile oluşturduğunu sorduk ve şöyle yanıtladık: ikisinin arasında çocuk kendi ortamını hazırlayabilir, örneğin çıktısını yönlendirebilir. İşte kanıtı. `ls data > list.txt` komutunu çalıştıran etkileşimsiz bir `bash`'i, çocukları da izleyerek (`-f`) ve yalnızca üç sistem çağrısını göstererek izliyoruz:

```
student@okan-lab:~/lab$ strace -f -o t2.txt -e trace=openat,dup2,execve bash -c "ls data >
    list.txt"
student@okan-lab:~/lab$ grep -E "list.txt|dup2|execve" t2.txt
15726 execve("/usr/bin/bash", ["bash", "-c", "ls data > list.txt"], 0x7fff5f5520e8 /* 23 vars
    */) = 0
15727 openat(AT_FDCWD, "list.txt", O_WRONLY|O_CREAT|O_TRUNC, 0666) = 3
15727 dup2(3, 1)                        = 1
15727 execve("/usr/bin/ls", ["ls", "data"], 0x55b277587230 /* 23 vars */) = 0
```

Her satırdaki ilk sayı PID'dir: $15726$ `bash`'tir, $15727$ ise onun çocuğu (`fork()`'un arkasındaki Linux sistem çağrısı olan `clone()` ile oluşturulmuştur; burada izlenmedi). Çocuk

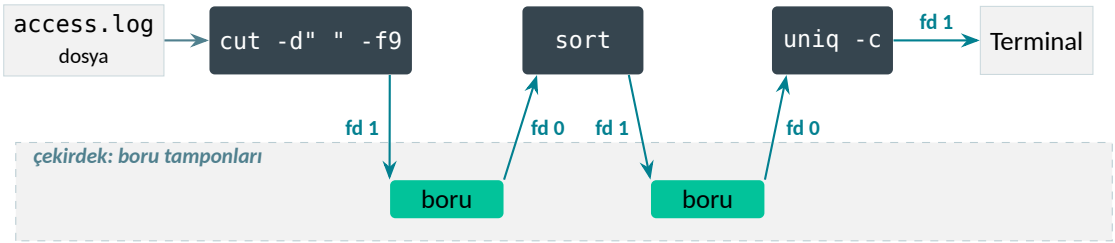
1. `list.txt`'yi yazmak için açar; `O_CREAT` dosyayı oluşturur, `O_TRUNC` içini boşaltır (önceki bölümdeki "> eski içeriği yok eder" durumu budur) ve çekirdek en küçük boş tanımlayıcıyı, $3$'ü döndürür;
2. `dup2(3, 1)` çağırır: $1$ numaralı tanımlayıcı $3$'ün kopyası olur, böylece standart çıktı artık dosyaya gider (kabuk ardından fazladan kalan $3$ numaralı tanımlayıcıyı kapatır);
3. `execve("/usr/bin/ls", …)` çağırır: program `ls` ile değiştirilir, ama tanımlayıcı tablosu sağ kalır. `ls` $1$ numaralı tanımlayıcıya — yani dosyaya — yazar ve yönlendirildiğini hiç bilmez.

Aynı biçimde `>>`, `O_TRUNC` yerine `O_APPEND` ile açar; `<` dosyayı salt okunur açıp $0$ numaralı tanımlayıcının üzerine kopyalar; `2>&1` ise düpedüz `dup2(1, 2)` çağrısıdır. Yönlendirme ne `ls`'in ne de başka bir komutun özelliğidir: onu kabuk, sıradan sistem çağrılarıyla yapar.

## 4 Borular

### 4.1 Süreçleri birbirine bağlamak

Yönlendirme bir süreci bir *dosyaya* bağlar. `kmt1 | kmt2` biçiminde yazılan bir **boru** (pipe) ise bir süreci başka bir *sürece* bağlar: `kmt1`'in standart çıktısı `kmt2`'nin standart girdisi olur. Borularla birleştirilmiş komutlar zincirine **boru hattı** (pipeline) denir.




**Şekil 3:** `cut -d" " -f9 logs/access.log | sort | uniq -c` boru hattı: çekirdek içindeki iki tamponla bağlanmış, aynı anda çalışan üç süreç. Her aşamanın standart hatası (fd $2$) yine terminale gider.

```
student@okan-lab:~/lab$ cut -d" " -f9 logs/access.log | sort | uniq -c
```

```
    148 200
     20 301
     34 304
     17 403
     57 404
     24 500
```

`cut` her satırdan $9$. alanı (HTTP durum kodunu) çıkarır, `sort` eşit kodları bir araya getirir, `uniq -c` her grubu sayar. Hiç geçici dosya oluşturulmaz: veri, çekirdeğe ait bellek tamponlarından akar. Her aşama kendi PID'i olan bağımsız bir süreçtir (3. hafta) ve hepsini kabuk başlatır.

## 4.2 Kabuk bir boruyu nasıl kurar?

Bir boru `pipe()` sistem çağrısıyla (güncel Linux'ta `pipe2()`) oluşturulur; bu çağrı *iki* tanımlayıcı döndürür: biri okuma, biri yazma için. İki aşamalı bir boru hattının izi:

```
student@okan-lab:~/lab$ strace -f -o t3.txt -e trace=pipe2,clone,dup2,execve bash -c "grep
    ERROR logs/system.log | wc -l"
13
student@okan-lab:~/lab$ cat t3.txt
15736 execve("/usr/bin/bash", ["bash", "-c", "grep ERROR logs/system.log | wc "...], 0
    x7ffe43f1a528 /* 23 vars */) = 0
15736 pipe2([3, 4], 0)                  = 0
15736 clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD,
    child_tidptr=0x7f6367df1a10) = 15737
15736 clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD,
    child_tidptr=0x7f6367df1a10) = 15738
15737 dup2(4, 1)                        = 1
15738 dup2(3, 0)                        = 0
15738 execve("/usr/bin/wc", ["wc", "-l"], 0x55c9a2ad4920 /* 23 vars */) = 0
15737 execve("/usr/bin/grep", ["grep", "ERROR", "logs/system.log"], 0x55c9a2ad4920 /* 23 vars
    */) = 0
```

1. `pipe2([3, 4], 0)`: `bash` ($15736$) bir boru alır: $3$ numaralı tanımlayıcı okuma ucu, $4$ yazma ucudur.
2. İki `clone()` çağrısı (`= fork()`) $15737$ ve $15738$ numaralı çocukları oluşturur; ikisi de $3$ ve $4$ numaralı tanımlayıcıları miras alır.
3. İlk çocuk `dup2(4, 1)` çağırır: standart çıktısı artık yazma ucudur. Ardından `grep`'i çalıştırır.
4. İkinci çocuk `dup2(3, 0)` çağırır: standart girdisi okuma ucudur. `wc`'yi çalıştırır.
5. Kullanılmayan uçlar kapatılır (`close()` izlenmedi) ve `bash` iki çocuğu da bekler (izin burada atlanan geri kalan satırları, iki çıkışı ve çekirdeğin bir çocuğun sonlandığını ebeveyne bildirdiği `SIGCHLD` sinyalini gösterir).

Bu çalıştırmada `wc`'nin `grep`'ten *önce* çalıştırıldığına dikkat edin: iki çocuk eşzamanlı çalışır ve kimin önce gideceğine zamanlayıcı (scheduler, 4. hafta) karar verir. İki süreç aynı anda sistem çağrısı yaptığında `strace` bir çağrıyı `<unfinished ...>` ve `<... resumed>` diye işaretlenmiş iki yarıda yazabilir; bu normaldir.

Bir boru `/proc` içinde de görünür: `ls -l /proc/self/fd | cat` komutunda `ls`'in $1$ numaralı tanımlayıcısı `1 -> pipe:[29942]` olarak gösterilir; bu, çekirdek içinde bir numarayla tanımlanan bir borudur (tam çıktı `code/out_pipes.txt` içinde).

**Dört sistem çağrısında yönlendirme ve borular**

`pipe()` kanalı oluşturur, `fork()` süreçleri oluşturur, `dup2()` kanalı (ya da `open()` ile açılmış bir dosyayı) $0$, $1$ veya $2$ numaralı tanımlayıcıya takar, `exec()` ise tesisatı farkında olmadan miras alan programı başlatır.

### 4.3 Boru sınırlı bir tampondur (5. hafta)

Bir borunun sınırlı bir kapasitesi vardır: Linux'ta varsayılan değer $64$ KiB $= 16 \times 4096 = 65\,536$ bayttır (on altı adet $4$ KiB'lık sayfa). Çekirdek üç kural uygular:

- **Tampon dolu:** `write()` bloke olur — okuyan veri alana kadar yazan uyutulur.
- **Tampon boş:** `read()` bloke olur — veri gelene kadar okuyan uyur.
- **Hiç yazan kalmadı ve tampon boş:** `read()` $0$ döndürür, bu da dosya sonu demektir; okuyan işini bitirir. **Hiç okuyan kalmadı:** yazan, kendisini sonlandıran `SIGPIPE` sinyalini alır.

Bu tam olarak 5. haftanın *sınırlı tampon* (bounded buffer, üretici–tüketici) problemidir ve çekirdek içinde çözülmüştür: tampon doluyken üretici, boşken tüketici beklemek zorundadır. İki deney bunun sonuçlarını gösterir:

```
student@okan-lab:~/lab$ time (sleep 2 | sleep 2)

real    0m2.005s
user    0m0.003s
sys     0m0.002s
student@okan-lab:~/lab$ yes | head -n 3
y
y
y
student@okan-lab:~/lab$ yes "MCMP117" | head -n 100000 | wc -l
100000
```

`sleep 2 | sleep 2` $4$ s değil $2{,}0$ s sürer: aşamalar *eşzamanlı* çalışır. `yes` sonsuza kadar `y` yazar, yine de boru hattı durur: üç satırdan sonra `head` çıkar, borunun okuyanı kalmaz ve `yes`'in bir sonraki `write()` çağrısı `SIGPIPE` sinyalini tetikler. (Sinyaller 13. haftanın konusudur.) Son boru hattı $100\,000$ satırı, yaklaşık $800$ kB'ı, iki adet $64$ KiB'lık tampondan sorunsuzca geçirir: veri üretildiği hızla tüketilir.

**Borular ve UNIX felsefesi**

Bell Labs'tan Doug McIlroy, 1964 tarihli bir notunda programların "bahçe hortumu gibi — veriyi başka bir biçimde işlemek gerektiğinde bir parça daha vidalayarak" bağlanması gerektiğini yazdı. Borular 1973'te UNIX'in 3. sürümünde, Ken Thompson `pipe()` sistem çağrısını ve kabuğa `|` gösterimini eklediğinde ortaya çıktı. McIlroy daha sonra ortaya çıkan üslubu şöyle özetledi: tek bir iş yapan ve onu iyi yapan programlar yazın; birlikte çalışan programlar yazın; metin akışlarını işleyen programlar yazın, çünkü bu evrensel bir arayüzdür. `grep`'in adı da ed metin düzenleyicisinin `g/re/p` ("global regular expression print") komutundan gelir.

## 5 Satır Aramak: `grep`

`grep [seçenekler] KALIP [dosyalar]`, kalıbı içeren satırları yazdırır.

```
student@okan-lab:~/lab$ grep ERROR logs/system.log | head -n 3
2026-10-12 09:12:00 ERROR failed to mount /dev/sdb1
2026-10-12 09:56:00 ERROR failed to mount /dev/sdb1
2026-10-12 09:07:00 ERROR failed to mount /dev/sdb1
student@okan-lab:~/lab$ grep -c ERROR logs/system.log
13
student@okan-lab:~/lab$ grep -i "TALK" docs/linux_quotes.txt
Talk is cheap. Show me the code.
student@okan-lab:~/lab$ grep -n "backup" logs/system.log | head -n 4
6:2026-10-12 09:55:00 INFO  backup finished
7:2026-10-12 09:06:00 INFO  backup started
10:2026-10-12 09:39:00 INFO  backup finished
29:2026-10-12 10:08:00 INFO  backup started
student@okan-lab:~/lab$ grep -v INFO logs/system.log | head -n 4
2026-10-12 09:11:00 WARN  high memory usage
2026-10-12 09:22:00 WARN  high memory usage
2026-10-12 09:12:00 ERROR failed to mount /dev/sdb1
2026-10-12 09:23:00 WARN  temperature 71C
student@okan-lab:~/lab$ grep -vc INFO logs/system.log
41
```

| Seçenek | Anlamı |
|---|---|
| `-i` | büyük ve küçük harf arasındaki farkı yok say |
| `-n` | her satırın önüne satır numarasını yaz |
| `-v` | tersine çevir: eşleş*meyen* satırları yazdır |
| `-c` | yalnızca eşleşen *satırların* sayısını yazdır (geçiş sayısını değil) |
| `-w` | eşleşme tam bir sözcük olmalı |
| `-r` | bir dizinin altındaki tüm dosyalarda (özyinelemeli) ara |
| `-l` | yalnızca eşleşme içeren dosyaların adlarını yazdır |

Seçenekler birleştirilebilir: `grep -vc INFO`, INFO olmayan satırları sayar: $41$. (İleride bunların $13$ ERROR ve $28$ WARN satırı olduğunu göreceğiz; $13 + 28 = 41$.) `system.log`'daki dakikalar bir saat içinde sıralı değildir; bu üretilmiş alıştırma verisidir, gerçek günlükler ise zaman sırasındadır.

## 5.1 Tam sözcükler ve tüm dizinler

```
student@okan-lab:~/lab$ grep "Week 1" docs/syllabus.txt
Week 1  Introduction and history
Week 10 File and directory operations
Week 11 Permissions, ownership and users
Week 12 Text processing and redirection
Week 13 Processes and job control
Week 14 Introduction to shell scripting
Week 15 Final exam
student@okan-lab:~/lab$ grep -w "Week 1" docs/syllabus.txt
Week 1  Introduction and history
student@okan-lab:~/lab$ grep -r "Project" projects
projects/beta/README.md:# Project beta
projects/alpha/README.md:# Project alpha
```

`Week 1` kalıbı `Week 10` … `Week 15` içinde de geçer. `-w` ile eşleşmenin önünde ya da arkasında harf, rakam veya alt çizgi bulunamaz. `-r` ile `grep`, `projects` altındaki her dosyada arar ve her satırın önüne dosya adını yazar; dosyalar alfabetik değil, dizindeki sırayla görünür.

## 5.2 Basit düzenli ifadeler

`grep`'in kalıbı bir **düzenli ifadedir** (regular expression, regex): dizgi kümelerini betimleyen küçük bir dil. Karakterlerin çoğu kendisiyle eşleşir; birkaçının özel anlamı vardır.

| Simge | Eşleştiği | Örnek | Örneğin şununla eşleşir |
|---|---|---|---|
| `^` | satırın başı | `^Week` | `Week` ile başlayan satırlar |
| `$` | satırın sonu | `s$` | `s` ile biten satırlar |
| `.` | herhangi *tek* bir karakter | `c.de` | `code`, `cxde` |
| `[ ]` | bir küme ya da aralıktan tek karakter | `[0-9]`, `[aeiou]` | `7`, `e` |
| `*` | önündeki öğe, 0 ya da daha çok kez | `[0-9]*` | boş, `4`, `2026` |
| `\.` | gerçek bir nokta (`\` özel anlamı kaldırır) | `5\.38` | yalnızca `5.38` |

```
student@okan-lab:~/lab$ grep "^Week [0-9] " docs/syllabus.txt
Week 1  Introduction and history
Week 2  Definitions, goals, components, structures
Week 3  OS services, system calls, processes
...
student@okan-lab:~/lab$ grep "s$" docs/syllabus.txt
Week 2  Definitions, goals, components, structures
Week 3  OS services, system calls, processes
Week 4  Threads and CPU scheduling basics
...
student@okan-lab:~/lab$ grep "c.de" docs/linux_quotes.txt
Talk is cheap. Show me the code.
student@okan-lab:~/lab$ grep -c "5.38" logs/access.log
39
student@okan-lab:~/lab$ grep -c "5\.38" logs/access.log
37
student@okan-lab:~/lab$ grep "5.38" logs/access.log | grep -v "5\.38"
10.0.6.45 - - [12/Oct/2026:15:53:47 +0300] "GET /courses/mcmp117 HTTP/1.1" 200 5438
10.0.2.17 - - [12/Oct/2026:15:38:02 +0300] "GET /login HTTP/1.1" 200 1357
student@okan-lab:~/lab$ grep ",ECT,[0-9]*,9[0-9]$" data/students.csv
2026001,Elif Yilmaz,ECT,96,99
2026015,Deniz Celik,ECT,95,96
student@okan-lab:~/lab$ grep "^2026-10-12 1[0-1]:.*ERROR" logs/system.log
2026-10-12 10:40:00 ERROR failed to mount /dev/sdb1
2026-10-12 10:35:00 ERROR failed to mount /dev/sdb1
2026-10-12 10:30:00 ERROR failed to mount /dev/sdb1
2026-10-12 10:03:00 ERROR failed to mount /dev/sdb1
```

- `^Week [0-9]` (sondaki boşluğa dikkat) satır başında `Week`, bir boşluk, tek bir rakam ve bir boşlukla eşleşir: tek basamaklı $1$–$9$. haftalar.
- Kaçışsız nokta klasik bir tuzaktır. `10.0.5.38` IP adresinin isteklerini `5.38` kalıbıyla saymak $39$ verir, ama doğru yanıt $37$'dir: nokta, `15:38` saatindeki `:` karakteriyle ve 5438 bayt sayısındaki 4 rakamıyla da eşleşti. Son `grep` boru hattı tam olarak bu iki fazla satırı gösterir.
- `,ECT,[0-9]*,9[0-9]$`, *final* notu (son alan) $90$ ile $99$ arasında olan ECT öğrencilerini bulur.
- `.*`, "herhangi sayıda herhangi karakter" demektir: `^2026-10-12 1[0-1]:.*ERROR`, $10$:00 ile $11$:59 arasındaki ERROR satırlarını bulur.

**Düzenli ifadeler joker karakter değildir**

Kabukta (10. hafta) `*` "herhangi bir dizgi", `?` ise "herhangi bir karakter" demektir. Düzenli ifadede `*` "önceki öğeyi tekrarla", `.` ise "herhangi bir karakter" demektir. Kabuğun `*`, `[ ]` ya da boşlukları `grep` görmeden önce genişletmeye çalışmaması için kalıbı her zaman tırnak içine alın. (`grep -E`, `+`, `?`, `|` ve gruplar içeren *genişletilmiş* düzenli ifadeleri etkinleştirir; bu hafta bunlara ihtiyacımız yok.)

## 6 Sıralamak ve Saymak: `sort` ve `uniq`

```
student@okan-lab:~/lab$ sort data/cities.txt | head -n 4
Ankara
Ankara
Antalya
Antalya
student@okan-lab:~/lab$ tail -n +2 data/students.csv | cut -d, -f5 | sort | head -n 3
100
35
36
student@okan-lab:~/lab$ tail -n +2 data/students.csv | cut -d, -f5 | sort -n | head -n 3
35
36
36
student@okan-lab:~/lab$ sort -t, -k5 -nr data/students.csv | head -n 3
2026010,Onur Dogan,ME,73,100
2026001,Elif Yilmaz,ECT,96,99
2026020,Emre Dogan,ME,64,98
```

Seçeneksiz `sort` satırları *karakter karakter* karşılaştırır; bu yüzden `100`, `35`'ten önce gelir (`1` karakteri `3`'ten küçüktür). `-n` ile sayıları karşılaştırır. Başlıca seçenekler:

| | |
|---|---|
| `-n` | sayısal sıra ($35 < 100$) |
| `-r` | ters sıra (en büyük önce) |
| `-t,` | alan ayırıcı virgüldür (varsayılan: boşluklar) |
| `-k5` | $5$. alana göre sırala ($5$. alandan satır sonuna kadar; `-k5,5` yalnızca $5$. alan demektir) |
| `-u` | her farklı satırı yalnızca bir kez yaz |
| `-o dosya` | sonucu *dosyaya* yaz; *dosya* girdinin kendisi olsa bile güvenlidir |

`tail -n +2` komutu bir dosyayı $2$. satırdan itibaren yazdırır ve `id,name,department,midterm,final` CSV başlığını atlar (sütunlar: numara, ad, bölüm, vize, final); unutursanız ne olacağı için bkz. Bölüm 11.

`uniq` *bitişik* yinelenen satırları kaldırır; `-c` her satırın önüne tekrar sayısını yazar, `-d` yalnızca tekrarlanan satırları yazdırır. `uniq` bir satırı yalnızca hemen önceki satırla karşılaştırdığından neredeyse her zaman `sort`'tan sonra gelmelidir:

```
student@okan-lab:~/lab$ uniq -c data/cities.txt | head -n 4
      1 Antalya
      1 Trabzon
      1 Konya
      1 Bursa
student@okan-lab:~/lab$ sort data/cities.txt | uniq -c
      2 Ankara
      6 Antalya
      5 Bursa
      5 Eskisehir
      1 Istanbul
      2 Izmir
      4 Konya
      5 Trabzon
student@okan-lab:~/lab$ sort data/cities.txt | uniq -c | sort -rn | head -n 3
      6 Antalya
      5 Trabzon
      5 Eskisehir
student@okan-lab:~/lab$ cut -d, -f2 data/students.csv | sort | uniq -d
Elif Yilmaz
Mehmet Aydin
Onur Dogan
Selin Yildiz
```

Sıralanmamış dosyada `uniq -c`, Antalya $6$ kez geçtiği hâlde `1 Antalya` bildirir: altı satır yan yana değildir. `sort`'tan sonra sayılar doğrudur ve toplamları tutar: $2+6+5+5+1+2+4+5=30$ satır. Şu bileşim

```
sort | uniq -c | sort -rn
```

"her birinden kaç tane, en sık olan önce" sorusunu yanıtlar ve bu haftanın en yararlı boru hattıdır. Eşit sayılar (Trabzon ve Eskisehir, her biri $5$) satırın geri kalanına göre sıralanır ve `-r` bu sırayı da ters çevirir. Son olarak `uniq -d`, `students.csv`'de dört adın iki kez geçtiğini gösterir: aynı adı taşıyan farklı öğrenciler (farklı numaralar).

# 7 Sütunlar, Sayımlar ve Karakterler: `cut`, `wc`, `tr`

## 7.1 `cut`

```
student@okan-lab:~/lab$ head -n 3 data/students.csv | cut -d, -f2
name
Elif Yilmaz
Zeynep Dogan
student@okan-lab:~/lab$ head -n 3 data/students.csv | cut -d, -f2,5
name,final
Elif Yilmaz,99
Zeynep Dogan,36
student@okan-lab:~/lab$ head -n 2 logs/access.log | cut -d" " -f1,7,9
```

```
10.0.1.10 /about.html 404
10.0.1.10 /api/grades 200
student@okan-lab:~/lab$ head -n 3 logs/system.log | cut -c12-16
09:00
09:11
09:22
student@okan-lab:~/lab$ head -n 3 logs/system.log | cut -d" " -f4



student@okan-lab:~/lab$ head -n 3 logs/system.log | tr -s " " | cut -d" " -f4-
network link up
high memory usage
high memory usage
```

`cut -d AYIRICI -f LİSTE`, her satırı tek bir `AYIRICI` karakterinde böler ve listelenen alanları (1'den sayılarak) tutar: `-f2,5` $2$. ve $5$. alanlar, `-f2-4` $2$–$4$. alanlar, `-f4-` $4$. alandan sona kadar. `cut -c 12-16`, $12$–$16$. karakterleri tutar; sabit genişlikli sütunlar için kullanışlıdır.

`system.log`'da düzey beş karaktere tamamlanmıştır (`INFO␣␣network`); bu yüzden `INFO` ile ileti arasında *iki* boşluk vardır. `cut` her boşluğu bir ayırıcı sayar, aralarında boş bir alan görür ve $4$. alan boştur — üç boş satırın nedeni budur. `tr -s " "` önce boşluk dizilerini tek bir boşluğa sıkıştırır.

## 7.2 wc

```
student@okan-lab:~/lab$ wc logs/system.log
 120  669 5207 logs/system.log
student@okan-lab:~/lab$ wc -l data/*
  30 data/cities.txt
  20 data/numbers.txt
  41 data/students.csv
  91 total
```

wc satır, sözcük ve bayt sayısını yazar: `system.log` $120$ satır, $669$ sözcük ve $5207$ bayttır. `-l`, `-w` ve `-c` bunlardan birini seçer; birden çok dosyada bir `total` (toplam) satırı eklenir. Satır sayısı, satır sonu karakterlerinin sayısıdır; bu yüzden sonunda satır sonu olmayan bir son satır sayılmaz.

## 7.3 tr

```
student@okan-lab:~/lab$ head -n 2 docs/linux_quotes.txt | tr a-z A-Z
TALK IS CHEAP. SHOW ME THE CODE.
GIVEN ENOUGH EYEBALLS, ALL BUGS ARE SHALLOW.
student@okan-lab:~/lab$ head -n 2 data/students.csv | tr "," ";"
id;name;department;midterm;final
2026001;Elif Yilmaz;ECT;96;99
student@okan-lab:~/lab$ echo "Hello, World!" | tr -d ",!"
Hello World
```

`tr KÜME1 KÜME2`, KÜME1'deki her karakteri KÜME2'deki karşılığıyla değiştirir; `tr -d KÜME` karakterleri siler; `tr -s KÜME` tekrarları sıkıştırır. `tr` sözcüklerle değil tek tek karakterlerle çalışır (`tr "cat" "dog"` her yerde c'yi d, a'yı o ve t'yi g yapar) ve yalnızca standart girdiyi okur: asla dosya adı almaz; bu yüzden `<` ya da bir boru kullanın.

# 8 Diğer Araçlar: `head`, `tail`, `tee`, `find`, `xargs`

## 8.1 Boru hatlarında `head` ve `tail`; `tee`

```
student@okan-lab:~/lab$ tail -n +2 data/students.csv | head -n 2
2026001,Elif Yilmaz,ECT,96,99
2026002,Zeynep Dogan,ME,95,36
student@okan-lab:~/lab$ head -n 120 logs/access.log | tail -n 1
10.0.3.24 - - [12/Oct/2026:12:53:47 +0300] "GET /index.html HTTP/1.1" 404 4270
student@okan-lab:~/lab$ grep " 500 " logs/access.log | tee errors_500.txt | wc -l
24
student@okan-lab:~/lab$ head -n 2 errors_500.txt
10.0.5.38 - - [12/Oct/2026:08:14:26 +0300] "GET /index.html HTTP/1.1" 500 4583
10.0.5.38 - - [12/Oct/2026:08:21:39 +0300] "GET /images/logo.png HTTP/1.1" 500 4233
student@okan-lab:~/lab$ wc -l errors_500.txt
24 errors_500.txt
```

`head -n N` ilk $N$ satırı, `tail -n N` son $N$ satırı, `tail -n +N` ise $N$. satırdan itibaren her şeyi tutar. Birlikte kullanıldığında `head -n 120 | tail -n 1` tam olarak $120$. satırı çıkarır. `tail -f dosya` dosyayı açık tutar ve eklenen yeni satırları yazdırır: bir günlüğü canlı izlemenin standart yolu (Ctrl+C ile durdurun).

`tee dosya` bir T bağlantısıdır (adını tesisatçının T parçasından alır): standart girdisini hem dosyaya hem de standart çıktısına kopyalar; böylece bir boru hattı ara sonucu kaydedip devam edebilir. `tee -a` sona ekler. 11. haftayla bir bağlantı: `sudo echo metin > /etc/dosya` başarısız olur, çünkü yönlendirmeyi `sudo` daha başlamadan, root hakları olmayan *sizin* kabuğunuz yapar; `echo metin | sudo tee /etc/dosya` ise çalışır, çünkü dosyayı root olarak çalışan `tee` açar.

## 8.2 `find`

`find NEREDE SINAMALAR`, NEREDE'nin altındaki bütün dizin ağacını dolaşır ve tüm sınamaları geçen her yolu yazdırır.

```
student@okan-lab:~/lab$ find docs -name "*.txt"
docs/notes_02.txt
docs/syllabus.txt
docs/notes_01.txt
docs/notes_03.txt
docs/linux_quotes.txt
docs/final report.txt
student@okan-lab:~/lab$ find . -type d -name src
./src
./projects/beta/src
./projects/alpha/src
student@okan-lab:~/lab$ find . -size +10k
./logs/access.log
student@okan-lab:~/lab$ find photos -iname "*.jp*g" | wc -l
14
student@okan-lab:~/lab$ touch -d "2026-09-01" docs/notes_01.txt
student@okan-lab:~/lab$ find docs -mtime +30
docs/notes_01.txt
```

| Sınama | Anlamı |
|---|---|
| `-name "*.txt"` | ad, joker karakterli kalıpla eşleşir (tırnakları bırakmayın!) |
| `-iname` | aynısı, harf büyüklüğüne bakmadan |
| `-type f`, `-type d` | normal dosya, dizin |
| `-size +10k`, `-size -10k` | $10$ KiB $= 10\,240$ bayttan büyük / küçük |
| `-mtime +30`, `-mtime -1` | $30$ günden daha önce / son $24$ saat içinde değiştirilmiş |

`-name "*.txt"` içindeki tırnaklar olmasaydı, kabuk `*.txt`'yi `find` başlamadan önce *şu anki* dizinde genişletirdi (10. hafta). Yalnızca `access.log` ($23\,604$ bayt) $10$ KiB'ı aşar. $14$ fotoğraf şunlardır: on iki `img0NN.jpg`, `IMG_014.JPG` ve `holiday.jpeg`; `img013.png` eşleşmez. Lab dosyalarının hepsi yeni oluşturulduğu için, `-mtime +30`'un bir şey bulabilmesi amacıyla `notes_01.txt`'ye `touch -d` ile eski bir değiştirilme zamanı verdik (10. hafta). `grep -r` gibi `find` da girdileri dizin sırasıyla listeler; sabit bir sıra gerekiyorsa çıktıyı boruyla `sort`'a verin.

**`find` hataları.** `find` okumasına izin verilmeyen bir dizinle karşılaştığında (11. hafta) bir hata bildirir — standart hata üzerinden; bu yüzden hata ayrıca atılabilir:

```
student@okan-lab:~/lab$ mkdir secret && chmod 000 secret
student@okan-lab:~/lab$ find . -name "*.md"
./projects/beta/README.md
./projects/alpha/README.md
find: ‘./’secret: Permission denied
student@okan-lab:~/lab$ find . -name "*.md" 2> /dev/null
./projects/beta/README.md
./projects/alpha/README.md
```

(`secret`'ın izinlerini `chmod 000` ile kaldırdık; sonrasında izinleri `chmod 755 secret` ile geri verin ve dizini `rmdir secret` ile silin.)

## 8.3 `-exec` ve `xargs`

```
student@okan-lab:~/lab$ find . -name "*.c" -exec wc -l {} \;
7 ./src/hello.c
1 ./projects/beta/src/main.c
1 ./projects/alpha/src/main.c
student@okan-lab:~/lab$ find . -name "*.c" | xargs wc -l
  7 ./src/hello.c
  1 ./projects/beta/src/main.c
  1 ./projects/alpha/src/main.c
  9 total
student@okan-lab:~/lab$ find docs -name "*.txt" | xargs wc -l
  1 docs/notes_02.txt
 15 docs/syllabus.txt
  1 docs/notes_01.txt
  1 docs/notes_03.txt
  5 docs/linux_quotes.txt
wc: docs/final: No such file or directory
wc: report.txt: No such file or directory
 23 total
```

```
student@okan-lab:~/lab$ find docs -name "*.txt" -print0 | xargs -0 wc -l
  1 docs/notes_02.txt
 15 docs/syllabus.txt
  1 docs/notes_01.txt
  1 docs/notes_03.txt
  5 docs/linux_quotes.txt
  2 docs/final report.txt
 25 total
```

- `-exec kmt {} \;`, bulunan her dosya için `kmt`'yi bir kez çalıştırır; `{}` dosyanın adıyla değiştirilir. `\;` komutun sonunu işaretler (ters eğik çizgi, kabuğun `;` karakterini yorumlamasını engeller).
- `xargs kmt`, standart girdisinden sözcükleri okur ve bunları `kmt`'ye *argüman* olarak verir: tek bir `wc` üç adı birden alır ve bir toplam yazar. Veri ile argüman arasındaki fark budur: `find … | wc -l` *adları* sayar; `find … | xargs wc -l` ise *dosyaların içindeki* satırları sayar.
- `xargs` varsayılan olarak boşluklarda böler; bu yüzden `final report.txt` var olmayan iki ada, `docs/final` ve `report.txt`'ye dönüştü ve toplam (23) dosyanın 2 satırını kaçırdı. `find -print0` her adı bir sıfır baytıyla bitirir (bu bayt bir dosya adında bulunamaz) ve `xargs -0` yalnızca orada böler: toplam 25.

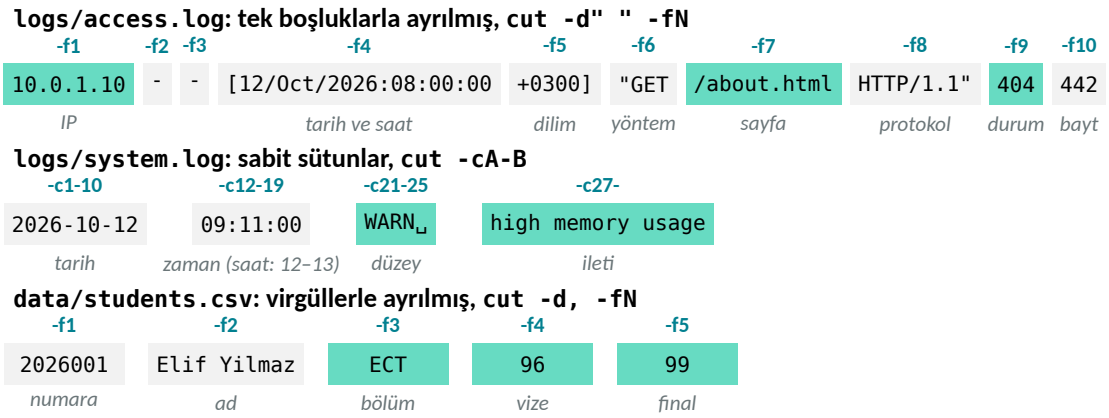
**Güvenlik: silmeden önce bakın**

`find … -exec rm {} \;` ve `find … | xargs rm`, listedeki her dosyayı sormadan ve çöp kutusu olmadan siler. Aynı komutu her zaman önce `rm` *olmadan* çalıştırın, listeyi okuyun ve `rm`'yi ancak ondan sonra ekleyin. Aynısı `rm -r` için de geçerlidir.

# 9 Tek Satırlık Boru Hatlarıyla Gerçek Analizler

## 9.1 Verinizi tanıyın

Bir boru hattı yazmadan önce birkaç satıra bakın (`head -n 3`) ve alanların nasıl ayrıldığına karar verin.




**Şekil 4:** Üç lab dosyasında alanların yerleri. `students.csv`'nin 1. satırı bir başlıktır.

Web sunucusu günlüğü yaygın olarak kullanılan *ortak günlük biçimini* (common log format) kullanır. 4. alan tarihi ve saati içerir; saati almak için iki noktalarda da kesebilirsiniz: `cut -d: -f2` `08` verir. HTTP durum

kodları beş sınıf oluşturur: `2xx` başarı (`200` OK), `3xx` yeniden yönlendirme (`301` kalıcı olarak taşındı, `304` değişmedi: önbellekteki kopyanızı kullanın), `4xx` istemci hataları (`403` yasak, `404` bulunamadı) ve `5xx` sunucu hataları (`500` iç sunucu hatası).

## 9.2 Yöntem: her seferinde bir aşama

```
student@okan-lab:~/lab$ cut -d" " -f9 logs/access.log | head -n 3
404
200
500
student@okan-lab:~/lab$ cut -d" " -f9 logs/access.log | sort | head -n 3
200
200
200
```

```
student@okan-lab:~/lab$ cut -d" " -f9 logs/access.log | sort | uniq -c | sort -rn
    148 200
     57 404
     34 304
     24 500
     20 301
     17 403
```

Her sayma sorusu aynı dört adımı izler:

1. Gereken sütunu **çıkarın** (`cut`); gerekirse önce ilgili satırları **süzün** (`grep`); `head` ile denetleyin.
2. **Gruplayın**: `sort` eşit değerleri bir araya getirir.
3. **Sayın**: `uniq -c`.
4. **Sıralayın**: `sort -rn`, ardından ilk $N$ için `head -n N`.

Altı aşamalı bir boru hattını asla tek seferde yazmayın: yukarı oka basın, bir aşama ekleyin, sonuca bakın, tekrarlayın. Sonuçları elinizden geldiğince denetleyin. Burada sayıların toplamı satır sayısını verir: $148+57+34+24+20+17=300$; başarılı isteklerin payı ise $148/300 \approx \%49{,}3$'tür.

## 9.3 access.log: sayfalar, ziyaretçiler, kırık bağlantılar

```
student@okan-lab:~/lab$ cut -d" " -f7 logs/access.log | sort | uniq -c | sort -rn | head -n 5
     41 /courses/mcmp117
     37 /about.html
     33 /index.html
     33 /images/logo.png
     32 /api/grades
student@okan-lab:~/lab$ cut -d" " -f1 logs/access.log | sort | uniq -c | sort -rn | head -n 3
     58 10.0.2.17
     56 10.0.4.31
     52 10.0.6.45
student@okan-lab:~/lab$ grep " 404 " logs/access.log | cut -d" " -f7 | sort | uniq -c | sort -
    rn | head -n 3
      9 /images/logo.png
      7 /index.html
      7 /about.html
```

Aynı tarif, 7. alanla (sayfa) ve 1. alanla (IP adresi). Üçüncü boru hattı önce `404` satırlarını süzer, sonra sayfayı çıkarır: en sık görülen kırık bağlantı `/images/logo.png`'dir (9 kez). `" 404 "` içindeki boşluklar önemlidir: onlar olmadan `4042` gibi bir bayt sayısı ya da bir IP parçası da eşleşebilirdi. Küçük verilerde eşitlikler sık görülür: `/index.html` ve `/images/logo.png`'nin ikisi de 33 istek almıştır.

### 9.4 system.log ve students.csv

```
student@okan-lab:~/lab$ grep -c ERROR logs/system.log
13
student@okan-lab:~/lab$ grep ERROR logs/system.log | cut -c12-13 | uniq -c
      3 09
      4 10
      1 12
      3 13
      2 14
student@okan-lab:~/lab$ grep WARN logs/system.log | cut -c27- | sort | uniq -c
     11 high memory usage
     17 temperature 71C
student@okan-lab:~/lab$ tail -n +2 data/students.csv | cut -d, -f3 | sort | uniq -c | sort -rn
     17 ECT
     11 ME
     10 CS
      2 EE
student@okan-lab:~/lab$ tail -n +2 data/students.csv | sort -t, -k5 -nr | head -n 3 | cut -d, -
    f2,3,5
Onur Dogan,ME,100
Elif Yilmaz,ECT,99
Emre Dogan,ME,98
student@okan-lab:~/lab$ grep ",ECT," data/students.csv | sort -t, -k4 -nr | head -n 3
2026017,Ipek Yilmaz,ECT,100,36
2026012,Arda Celik,ECT,98,63
2026006,Mehmet Aydin,ECT,98,85
```

- **Saat başına hatalar.** Burada `uniq -c`, `sort` olmadan çalışır; çünkü günlük zaten saat sırasındadır, dolayısıyla eşit saatler bitişiktir. Hata olmayan saatler (11 ve 15) yalnızca eksiktir: `uniq` orada olmayanı sayamaz. Denetim: $3 + 4 + 1 + 3 + 2 = 13$.
- **Uyarılar.** `cut -c27-` iletiyi tutar; $11 + 17 = 28$ WARN satırı.
- **Bölümler.** $17 + 11 + 10 + 2 = 40$ öğrenci $= 41$ satır eksi başlık.
- **En iyi final notları.** Önce bütün satırları 5. alana göre sıralayın, gösterilecek sütunları en sonda kesin; önce kesmek, sıralama yaptığınız sütunu kaybettirir.

## 10 Komut Başvuru Tablosu

| Komut / sözdizimi | Anlamı |
|---|---|
| `kmt > f` | standart çıktı `f`'ye (önce oluştur ya da boşalt) |
| `kmt >> f` | standart çıktı `f`'nin sonuna eklenir |

| Komut / sözdizimi | Anlamı |
|---|---|
| `kmt < f` | standart girdi `f`'den |
| `kmt 2> f`, `2>> f` | standart hata `f`'ye (üzerine yaz, sona ekle) |
| `kmt > f 2>&1`, `kmt &> f` | iki akış da `f`'ye |
| `kmt 2> /dev/null` | hata iletilerini at |
| `kmt1 \| kmt2` | `kmt1`'in standart çıktısı `kmt2`'nin standart girdisine |
| `set -o noclobber` | > var olan dosyaların üzerine yazmayı reddeder (>\| zorlar) |
| `grep KALIP dosyalar` | `KALIP` düzenli ifadesini içeren satırlar |
| `-i -n -v -c -w -r -l` | harf büyüklüğünü yok say, satır numaraları, tersine çevir, say, tam sözcük, özyinelemeli, yalnızca dosya adları |
| `sort` | satırları sırala; `-n` sayısal, `-r` ters, `-t,` ayırıcı, `-k5` anahtar alan, `-u` benzersiz, `-o f` çıktı dosyası |
| `uniq` | bitişik tekrarları birleştir; `-c` say, `-d` yalnızca tekrarlananlar |
| `cut -d, -f2,5` | virgülle ayrılmış satırların $2.$ ve $5.$ alanları; `-f4-` $4.$ alandan sona |
| `cut -c12-16` | $12$–$16.$ karakterler |
| `wc` | satır, sözcük, bayt; `-l`, `-w`, `-c` |
| `tr a-z A-Z` | karakterleri dönüştür (yalnızca stdin okur); `-d` sil, `-s` sıkıştır |
| `head -n N`, `tail -n N` | ilk / son $N$ satır |
| `tail -n +N` | $N.$ satırdan sona kadar ($N = 2$ ile başlığı atla) |
| `tail -f f` | büyüyen bir dosyayı izle (Ctrl+C durdurur) |
| `tee f`, `tee -a f` | stdin'i `f`'ye ve stdout'a kopyala |
| `find DİZİN -name "*.c"` | `DİZİN` altında adı eşleşen dosyalar; `-iname` harf büyüklüğüne bakmaz |
| `-type f\|d, -size +10k, -mtime +30` | dosya türü, boyut, gün olarak yaş |
| `-exec kmt {} \;` | bulunan her dosyada `kmt`'yi çalıştır |
| `-print0 \| xargs -0 kmt` | adları (boşluklu olsalar bile) `kmt`'ye argüman olarak ver |
| `strace -f -e trace =… kmt` | `kmt`'nin ve çocuklarının sistem çağrılarını göster |

## 11 Sık Yapılan Hatalar ve Anlamları

Aşağıdakilerin hepsi bilerek üretildi (`code/demo_mistakes.sh`; tırnaksız `grep`'in yazdırdığı $15$ satır kısaltıldı):

```
student@okan-lab:~/lab$ cp data/cities.txt c.txt
student@okan-lab:~/lab$ sort c.txt > c.txt
student@okan-lab:~/lab$ wc -l c.txt
0 c.txt
student@okan-lab:~/lab$ grep user login logs/system.log
grep: login: No such file or directory
logs/system.log:2026-10-12 09:44:00 INFO  user login
...
logs/system.log:2026-10-12 14:43:00 INFO  user login
logs/system.log:2026-10-12 14:27:00 INFO  user login
student@okan-lab:~/lab$ grep "user login" logs/system.log | wc -l
15
student@okan-lab:~/lab$ ls data > wc
student@okan-lab:~/lab$ ls
README.txt  c.txt  docs  photos    scripts  wc
backup      data   logs  projects  src
```

```
student@okan-lab:~/lab$ cat wc
cities.txt
numbers.txt
students.csv
student@okan-lab:~/lab$ sort -t, -k5 -n data/students.csv | head -n 2
id,name,department,midterm,final
2026032,Onur Dogan,ECT,41,35
```

**`sort c.txt > c.txt` boş bir dosya bırakır.** Kabuk yönlendirmeyi *önce* yapar: `c.txt`'yi $0$ bayta kırpar, sonra boş bir dosya okuyan `sort`'u başlatır. Aynısı her `kmt f > f` için olur. Yeni bir dosyaya yazıp adını değiştirin (`sort c.txt > c2.txt && mv c2.txt c.txt`) ya da `sort -o c.txt c.txt` kullanın.

**`grep: login: No such file or directory`.** Tırnaklar olmadığında `grep`, `user` kalıbını iki ayrı dosyada arar: `login` ve `logs/system.log`. Boşluk içeren kalıpları tırnak içine alın: `grep "user login" logs/system.log`.

**`wc` adlı bir dosya ortaya çıkar.** `ls data > wc`, `wc` adlı bir dosyaya yönlendirir; `wc`'yi çalıştırmaz. Boruyu kullanın: `ls data | wc -l`.

**Başlık satırı sonuçta görünür.** `id,name,…` bir sayı değildir; bu yüzden `sort -n` onu $0$ sayar ve başa koyar. `tail -n +2` ile atlayın.

**`uniq -c` hep $1$ sayar.** Girdi sıralanmamıştı; `sort | uniq -c` kullanın.

**`2>&1` yazdığınız hâlde hatalar görünüyor.** Sıra `2>&1 > f` idi; `> f 2>&1` yazın (Şekil 2).

**`cut` boş satırlar yazdırır.** Ayırıcı art arda iki kez geçiyor (tamamlanmış sütunlar); önce `tr -s " "` ile sıkıştırın ya da `cut -c` kullanın.

**`xargs`: adın yarısı için `No such file or directory`.** Bir dosya adı boşluk içeriyor; `find … -print0 | xargs -0 …` kullanın.

**`find`'dan `Permission denied` satırları.** Okuma izniniz olmayan dizinler (11. hafta). Önemli değillerse `2> /dev/null` ekleyin.

**Boru hattı takılmış gibi görünüyor.** Dosya argümanı olmayan bir komut klavyeden girdi bekler (ör. tek başına `sort` ya da dosyasız `grep ERR0R`). Ctrl+D (girdi sonu) ya da Ctrl+C'ye basın.

**`tr: extra operand`.** `tr` dosya adı almaz; `tr a-z A-Z < dosya` kullanın.

## 12 Lab Föyü

**Hazırlık:** `bash mcmp117_lab_setup.sh`, ardından `cd ~/lab`. Her alıştırma için tek bir komut satırı (bir boru hattı tek satır sayılır) ve çıktısını yazın. Puanlar köşeli parantez içindedir, toplam $100$. Çözümler Bölüm 17'dadır; önce kendiniz deneyin!

### A Bölümü --- Yönlendirme (20 puan)

**A1** [4] `data/` içindeki dosyaların listesini `list.txt`'ye kaydedin, sonra `docs/` listesini aynı dosyanın sonuna ekleyin. `list.txt` kaç satırdır?

**A2** [4] `ls data nosuch` komutunu, liste `ok.txt`'ye, hata iletisi `err.txt`'ye gidecek ve ekranda hiçbir şey görünmeyecek biçimde çalıştırın.

**A3** [4] `logs/system.log`'un yalnızca satır sayısını, dosya adı olmadan yazdırın.

**A4** [4] Önce tahmin edin, sonra denetleyin: `ls data nosuch > both.txt 2>&1` ve `ls data nosuch 2>&1 > both.txt` komutlarından hangisi ekrana bir şey yazar ve neden?

**A5** [4] `mkdir secret && chmod 000 secret` ile okunamayan bir dizin oluşturun. `find . -name "*.md"` komutu artık bir hata yazdırıyor. Komutu yalnızca sonuçlar görünecek biçimde değiştirin.

### B Bölümü --- grep (22 puan)

**B1** [4] `logs/system.log` kaç ERROR satırı içeriyor?

**B2** [4] `system.log`'un INFO satırı *olmayan* ilk $3$ satırını satır numaralarıyla gösterin.

**B3** [4] `access.log`'daki kaç istek $404$ durum kodunu aldı?

**B4** [4] Hangi öğrenciler EE bölümündedir?

**B5** [6] `10.0.5.38` IP adresinden kaç istek geldi? Noktaların başka karakterlerle eşleşemeyeceğinden ve yalnızca ilk alanın eşleşebileceğinden emin olun.

### C Bölümü --- Tek satırlık boru hatları (36 puan, her biri 6)

**C1** Her bölümde kaç öğrenci var (başlık hariç)?

**C2** Final notu en yüksek $3$ öğrencinin adlarını ve final notlarını yazdırın.

**C3** Durum kodu başına istekleri, en sık olan önce gelecek biçimde sayın.

**C4** En çok istenen $3$ sayfa hangileridir?

**C5** En çok $404$ hatasını hangi sayfa üretti ve kaç tane?

**C6** `system.log` her saatte kaç ERROR satırı içeriyor?

### D Bölümü --- Zorlu sorular (22 puan)

**D1** [5] `students.csv`'de hangi öğrenci adları birden çok kez geçiyor?

**D2** [5] Tek bir komut satırında, durumu $500$ olan bütün satırları `errors_500.txt`'ye kaydedin *ve* kaç tane olduklarını yazdırın.

**D3** [6] `find` ve `xargs` ile `docs/` altındaki tüm `.txt` dosyalarının satırlarını, `final report.txt` dâhil, sayın. Toplam nedir?

**D4** [6] `access.log`'da kaç farklı IP adresi görünüyor? En az isteği hangi adres yaptı ve kaç tane?

## 13 Özet

**Yedi cümlede 12. hafta**

1. Her süreç üç açık dosya tanımlayıcıyla başlar: $0$ stdin, $1$ stdout, $2$ stderr — 2–3. haftalarda `write(1, …)` çağrısının kullandığı aynı $1$ numaralı tanımlayıcı.

2. Yönlendirme (`> >> < 2> 2>&1 &> /dev/null`) bu tanımlayıcıların nereyi gösterdiğini değiştirir; kabuk bunu `fork()` ile `exec()` arasında `open()` ve `dup2()` ile yapar, böylece program bunu hiç bilmez.
3. Yönlendirmeler soldan sağa uygulanır: `> f 2>&1` iki akışı da yakalar, `2>&1 > f` yakalamaz.
4. `pipe()` ile oluşturulan `kmt1 | kmt2` borusu, eşzamanlı çalışan iki süreci, 5. haftanın sınırlı tamponu gibi davranan bir çekirdek tamponuyla bağlar.
5. Filtreler dosyaları ya da stdin'i okur ve stdout'a yazar: `grep` (`^ $ . [ ] *` ile), `sort`, `uniq`, `cut`, `wc`, `tr`, `head`, `tail`, `tee`.
6. `find` ağaçta ad, tür, boyut ve yaşa göre arar; `-exec` ya da `xargs` sonuçlar üzerinde işlem yapar, boşluklu adlar için `-print0 | xargs -0`.
7. Sayma soruları tek bir tarifi izler: çıkar $\rightarrow$ `sort` $\rightarrow$ `uniq -c` $\rightarrow$ `sort -rn` $\rightarrow$ `head`; her seferinde bir aşama eklenerek kurulur ve toplamlarla denetlenir.

## 14 Sözlük

**Dosya tanımlayıcı (file descriptor)** Bir sürecin açık dosyalar tablosunda sıra numarası olan küçük tamsayı; $0$, $1$, $2$ standart akışlardır.

**Standart girdi / çıktı / hata** $0$, $1$, $2$ numaralı tanımlayıcılardaki akışlar; varsayılan olarak klavye ve ekran.

**Yönlendirme (redirection)** Bir standart akışın terminal yerine bir dosyayı göstermesini sağlamak (>, >>, <, 2>).

**`dup2(eski, yeni)`** `yeni` tanımlayıcıyı `eski`'nin kopyası yapan sistem çağrısı.

**Boru (pipe)** Okuma ve yazma ucu olan, bir sürecin çıktısını başka bir sürecin girdisine bağlayan çekirdek tamponu.

**Boru hattı (pipeline)** Borularla bağlanmış, eşzamanlı çalışan komutlar.

**`SIGPIPE`** Okuyanı olmayan bir boruya yazan sürece gönderilen sinyal; varsayılan olarak süreci sonlandırır.

**Filtre (filter)** Girdisini dosyalardan ya da stdin'den okuyan ve sonucunu stdout'a yazan program.

**Düzenli ifade (regular expression)** Dizgi kümelerini betimleyen, `grep`'in kullandığı kalıp dili.

**`/dev/null`** Kendisine yazılan her şeyi atan aygıt dosyası.

## 15 Kendiniz Deneyin

### Daha fazla deney

1. Önce `ls -l /proc/self/fd | cat`, sonra `ls -l /proc/self/fd 2>&1 | cat` komutunu çalıştırın. Her durumda hangi tanımlayıcılar borudur?
2. `ls docs` ve `ls docs | cat` komutlarını çalıştırın. Birincisi neden `'final report.txt'` adını tırnaklı ve sütunlar hâlinde yazıyor da ikincisi yazmıyor?

3. Kendi kurduğunuz üç aşamalı bir boru hattında `strace -f -o t.txt -e trace=pipe2,dup2,execve bash -c "..."` kullanın. Kaç `pipe2()` çağrısı görüyorsunuz? (Aşama sayısından bir eksik.)
4. Bir terminalde `tail -f logs/system.log` çalıştırın; ikinci bir terminalde `echo "2026-10-12 15:00:00 ERROR test" >> logs/system.log` çalıştırın. Birinci terminalde ne olur?
5. `access.log`'da en çok isteğin geldiği saati bulun (ipucu: `cut -d: -f2`). Bu üretilmiş dosya için yanıt neden şaşırtıcıdır?

## 16 Tekrar Soruları

### Kavramsal

1. Standart girdi, standart çıktı ve standart hata nedir? Hangi dosya tanımlayıcılarını kullanırlar ve standart hata neden standart çıktıdan ayrıdır?
2. Kabuğun `sort data/cities.txt > sorted.txt` komutunu çalıştırmak için ne yaptığını adım adım (sistem çağrılarıyla) açıklayın.
3. `kmt 2>&1 > f` neden hata iletilerini yine ekrana yazar?
4. `fork()` ve `exec()`'in ayrı çağrılar olmasının yönlendirmeyi neden kolaylaştırdığını açıklayın.
5. Boru sınırlı bir tampondur. Yazan okuyandan hızlıysa ne olur? Okuyan erken çıkarsa?
6. `uniq -c`'nin girdisi neden normalde sıralanmış olmalıdır?
7. `find docs -name "*.txt" | wc -l` ile `find docs -name "*.txt" | xargs wc -l` arasındaki fark nedir?
8. `*` karakterinin `ls *.txt` içindeki ve `grep "[0-9]*"` içindeki farklı anlamlarını açıklayın.

### Sayısal

9. Bir boru tamponu $4096$ baytlık $16$ sayfa tutar. Yazan bloke olmadan önce tampona `MCMP117\n` biçiminde (her biri $8$ bayt) kaç satır sığar?
10. `access.log`'da $300$ isteğin $148$'i $200$, $57$'si $404$ döndürdü; $17$ istek de $403$ döndürdüğüne göre isteklerin yüzde kaçı istemci hatasıdır (4xx)?
11. `grep -c "5.38"` $39$ satır, `grep -c "5\.38"` ise $37$ satır sayar. Kaç satır yalnızca kaçışsız nokta yüzünden eşleşti?
12. Bir boru hattının $k$ aşaması var. Kabuğun en az kaç sürece, kaç `pipe()` ve kaç `dup2()` çağrısına ihtiyacı vardır (dosyalara yönlendirmeler hariç)?

### Sayısal soruların yanıtları

9. $16 \times 4096 = 65\,536$ bayt; $65\,536/8 = 8192$ satır.
10. $57 + 17 = 74$ istemci hatası; $74/300 \approx \%24{,}7$.

11. $39 - 37 = 2$ satır (15:38 saati ve 5438 bayt sayısı).

12. $k$ süreç, $k-1$ boru ve $2(k-1)$ dup2() çağrısı: her boru, yazanında bir dup2() ($1$ numaralı tanımlayıcıya) ve okuyanında bir dup2() ($0$ numaralı tanımlayıcıya) gerektirir. $k = 2$ için: $2$ süreç, $1$ boru, $2$ dup2() çağrısı; tıpkı Bölüm 4'teki izde olduğu gibi.

## 17 Lab Föyü Çözümleri

Tüm çözümler yeni oluşturulmuş bir labda çalıştırıldı; `code/lab_solutions.sh` betiği hepsini çalıştırır ve tam gerçek çıktısı `code/out_lab_solutions.txt` içindedir. Öğretici bölümlerde zaten görünen çıktılar özetlenmiştir. Başka doğru çözümler de vardır; önemli olan doğru çıktı ve makul bir yöntemdir.

| | Komut satırı | Beklenen çıktı / yanıt |
|---|---|---|
| **A1** | `ls data > list.txt`<br>`ls docs >> list.txt`<br>`wc -l list.txt` | `9 list.txt`: `data/`'dan $3$ ad ve `docs/`'tan görünen $6$ ad (`.settings` gizlidir). |
| **A2** | `ls data nosuch > ok.txt 2> err.txt`<br>`cat err.txt` | Ekranda hiçbir şey yok; `err.txt` şunu içerir: `ls: cannot access 'nosuch': No such file or directory`. |
| **A3** | `wc -l < logs/system.log` | `120` |
| **A4** | `ls data nosuch > both.txt 2>&1`<br>`ls data nosuch 2>&1 > both.txt` | Hatayı yalnızca ikincisi yazdırır: `2>&1`, $1$ numaralı tanımlayıcıyı o henüz terminali gösterirken kopyalar (Şekil 2). |
| **A5** | `find . -name "*.md" 2> /dev/null` | `./projects/beta/README.md` ve `./projects/alpha/README.md`, hata satırı yok. |
| **B1** | `grep -c ERROR logs/system.log` | `13` |
| **B2** | `grep -vn INFO logs/system.log \| head -n 3` | $2.$, $3.$ (WARN high memory usage) ve $13.$ satırlar (ERROR failed to mount /dev/sdb1). |
| **B3** | `grep -c " 404 " logs/access.log` | `57` |
| **B4** | `grep ",EE," data/students.csv` | Deniz Aydin ve Ece Ozturk (`2026011`, `2026031`). |
| **B5** | `grep -c "^10\.0\.5\.38 " logs/access.log` | 37. Noktalar kaçışlı, kalıp satır başına bağlı ve ardından bir boşluk geliyor; bu yüzden ne `15:38`, ne `5438` ne de `10.0.5.380` eşleşebilir. |
| **C1** | `tail -n +2 data/students.csv \| cut -d, -f3 \| sort \| uniq -c` | CS $10$, ECT $17$, EE $2$, ME $11$. |
| **C2** | `tail -n +2 data/students.csv \| sort -t, -k5 -nr \| head -n 3 \| cut -d, -f2,5` | Onur Dogan $100$, Elif Yilmaz $99$, Emre Dogan $98$. |
| **C3** | `cut -d" " -f9 logs/access.log \| sort \| uniq -c \| sort -rn` | $200$: $148$, $404$: $57$, $304$: $34$, $500$: $24$, $301$: $20$, $403$: $17$ (toplam $300$). |
| **C4** | `cut -d" " -f7 logs/access.log \| sort \| uniq -c \| sort -rn \| head -n 3` | `/courses/mcmp117` $41$, `/about.html` $37$, `/index.html` $33$ — `/images/logo.png` ($33$) ile eşit; eşitlik fark edildiyse ikisi de kabul edilir. |

| | Komut satırı | Beklenen çıktı / yanıt |
|---|---|---|
| **C5** | `grep " 404 " logs/access.log \| cut -d" " -f7 \| sort \| uniq -c \| sort -rn \| head -n 1` | `/images/logo.png`, 9 kez. |
| **C6** | `grep ERROR logs/system.log \| cut -c12-13 \| uniq -c` | 09: 3, 10: 4, 12: 1, 13: 3, 14: 2 (11 ve 15: yok; toplam 13). |
| **D1** | `cut -d, -f2 data/students.csv \| sort \| uniq -d` | Elif Yilmaz, Mehmet Aydin, Onur Dogan, Selin Yildiz. Başlık yinelenemez, bu yüzden çıkarılması gerekmez. |
| **D2** | `grep " 500 " logs/access.log \| tee errors_500.txt \| wc -l` | 24; `errors_500.txt` 24 satırı tutar. |
| **D3** | `find docs -name "*.txt" -print0 \| xargs -0 wc -l` | Toplam 25 satır (düz `xargs` ile boşluklu ad bozulur ve toplam 23 olur). |
| **D4** | `cut -d" " -f1 logs/access.log \| sort -u \| wc -l`<br>`cut -d" " -f1 logs/access.log \| sort \| uniq -c \| sort -n \| head -n 1` | 6 adres; en az isteği `10.0.5.38` yaptı, 37. |

**Puanlama ipuçları.** Doğru çıktı veren, çalışan tek bir komut satırına tam puan; çıktı doğru ama birden çok komutla ya da geçici dosyalarla elde edilmişse (alıştırma dosya istemiyorsa) yarım puan; elle sayılarak bulunan doğru bir sayıya puan verilmez.

## 18 İleriye Bakış: 13. Hafta

- **13. hafta: Süreçler ve İş Denetimi.** Bugün kabuk bizim için süreçler oluşturdu ve onları borularla bağladı. Gelecek hafta süreçleri dışarıdan izliyoruz: `ps`, `pstree` ve `top` (PID'ler, ebeveyn PID'leri ve 3. haftadaki süreç durumları), ön plan ve arka plan işleri (`&`, `jobs`, `fg`, `bg`, Ctrl+Z, Ctrl+C), sinyaller (`kill`, `SIGTERM` ile `SIGKILL` — ve bugün `yes`'i durduran `SIGPIPE`), `nice` ile öncelikler (5. hafta) ve kendi gözünüzle görebileceğiniz bir zombi süreç.
- **14. hafta:** kabuk betikleri; bugünkü boru hatları otomatik raporların yapı taşlarına dönüşecek.

### İleri okuma (isteğe bağlı)

- A. Silberschatz, P. B. Galvin ve G. Gagne, *Operating System Concepts*, 10. baskı, Wiley: Kısım 2.2 (kullanıcı arayüzleri, komut yorumlayıcısı), Kısım 3.3 (`fork()` ve `exec()` ile süreç oluşturma) ve Kısım 3.7.4 (Pipes).
- R. H. ve A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*:
  - Bölüm 5, "Interlude: Process API" (Kısım 5.4, bir kabuğun çıktıyı `fork()` ile `exec()` arasında nasıl yönlendirdiğini gösterir): `https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-api.pdf`
  - Bölüm 39, "Interlude: Files and Directories" (dosya tanımlayıcılar, `dup()`): `https://pages.cs.wisc.edu/~remzi/OSTEP/file-intro.pdf`
- GNU Bash kılavuzu, "Redirections": `https://www.gnu.org/software/bash/manual/html_node/Redirections.html`

- GNU Coreutils kılavuzu (`sort`, `uniq`, `cut`, `tr`, `wc`, `head`, `tail`, `tee`): https://www.gnu.org/software/coreutils/manual/coreutils.html